The Omnicron Coproduct Partition

A First-Principles Set-Theoretic and Categorical Foundation for Sparse OMI-Lisp Blackboard Composition

Status: Authoritative integration draft
Primary construction: tagged coproduct of independent sparse knowledge boards
Runtime carrier: shared 256-position OMI-Lisp plane
Identity carrier: ordered 32-bit CAR/CDR coordinates
Local resolver: CONS
Equivalence maintenance: optional Union-Find after lawful identification
Projection rule: visible coordinate does not replace full relation identity

---

0. Core Claim

OMI-Lisp earns its shared character encoding by receiving finite, bounded, origin-preserving contributions from independent `.o` knowledge sources.

Each source contributes:

```
one source identity

one typed scope

one sparse 256-bit board

one full CAR coordinate

one full CDR coordinate

one resolver profile

one provenance tag
```

The contributions enter the shared runtime through a coproduct:

$$B = \bigsqcup_{i \in I} B_i$$

where each B_i is one origin-tagged contribution board.

CONS then resolves compatible contributions into the shared 256-position OMI-Lisp blackboard while preserving:

```
which source contributed each position

which CAR coordinate carried it

which CDR continuation selected it

which resolver produced the visible coordinate

which projective or affine carrier exposed it
```

The central law is:

```
Coproduct preserves origin.

Partition preserves disjoint ownership.

CONS composes compatible contributions.

Projection exposes a shared coordinate.

Validation determines lawful identification.

Union-Find records identification only after validation.
```

---

1. Why Coproduct Is the Primary Construction

Suppose two knowledge sources contribute sets:

$$A$$

and:

$$B$$

If they are already disjoint:

$$A \cap B = \varnothing$$

then their ordinary union is also a disjoint union:

$$A \sqcup B = A \cup B$$

But independent knowledge sources may use identical local coordinates.

For example:

```
RULES.o claims local position 0x48

FACTS.o also claims local position 0x48
```

The two contributions must not collapse merely because their visible position is equal.

The categorical coproduct preserves origin by tagging:

$$A \sqcup B = (A \times \{0\}) \cup (B \times \{1\})$$

Thus:

$$(0x48, \mathrm{RULES}) \ / \ (0x48, \mathrm{FACTS}) =$$

even though both project to:

```
0x48
```

This is exactly the blackboard requirement.

```
Same visible coordinate
does not mean same contribution.

Same byte
does not mean same authority.

Same projection
does not mean same identity.
```

---

2. The OMI Coproduct Object

Let I be the set of contributing `.o` knowledge sources.

Typical members include:

```
RULES.o

FACTS.o

COMBINATORS.o

CLOSURES.o

additional remote .o modules
```

Each source $i \in I$ contributes a board:

$$B_i \subseteq \{0, \ldots, 255\}$$

or equivalently one 256-bit predicate:

$$\chi_i : \{0, \ldots, 255\} \to \{0, 1\}$$

The full origin-preserving blackboard is:

$$\mathcal{B} = \bigsqcup_{i \in I} B_i$$

An element of $\mathcal{B}$ is not merely a byte.

It is:

$$(i, p)$$

where:

```
i
source identity

p
visible byte-plane coordinate
```

The runtime form is:

```
(contribution
  (origin . RULES.o)
  (coordinate . 0x48))
```

not merely:

```
0x48
```

---

3. The Shared 256-Position OMI-Lisp Plane

The visible OMI-Lisp plane is:

$$P = \{0, \dots, 255\}$$

It is a bounded projection surface.

Its locked carrier geometry is:

```
0x00–0x1F
low affine-control coordinate field

0x20–0x7F
low affine readable space

0x80–0x9F
high projective-control mirror field

0xA0–0xFF
high projective readable space
```

The high mirror is:

$$m(x) = x \oplus 0x80$$

with:

$$m(m(x)) = x$$

Thus the complete plane is a reversible low/high carrier pair.

The low Omicron gauge and its earned notation boundary remain distinct from the high projective mirror. The Omicron character doctrine defines the low `0x00..0x7F` gauge, its four 32-position sectors, the `US → SP` tangent, and the readable `?0_o` branch.

---

4. Full Relation Space Versus Visible Plane

The visible plane has 256 positions.

The full CAR and CDR coordinate domains are:

$$CAR \in \{0, \ldots, 2^{32} - 1\}$$

$$CDR \in \{0, \ldots, 2^{32} - 1\}$$

Therefore the conceptual ordered relation space is:

$$R = CAR_{32} \times CDR_{32}$$

with:

$$|R| = 2^{64}$$

The visible OMI-Lisp plane is a projection:

$$\pi : R \to P$$

Many full relations may project to the same byte coordinate:

$$\pi(c_1, d_1) = \pi(c_2, d_2)$$

while:

$$(c_1, d_1) \ / \ (c_2, d_2) =$$

Therefore:

```
the visible coordinate is not the relation identity
```

The canonical relation identity must retain:

```
ordered CAR

ordered CDR

origin tag

scope

resolver profile

board identity

projective/affine carrier profile
```

---

5. CAR, CDR, and CONS

The canonical ordered relation is:

```
CAR = omi---imo
CDR = ΔUS/?O_oΔ
```

with:

$$CAR \in \mathbb{W}_{32}$$

$$CDR \in \mathbb{W}_{32}$$

where $\mathbb{W}_{32}$ is the 32-bit word domain.

CONS computes:

$$CONS(CAR, CDR) = (\text{resolved word}, \text{visible coordinate})$$

The result must contain both:

```
full resolved relation

bounded blackboard projection
```

Suggested form:

```
(cons-coordinate
  (car . 0x12345678)
  (cdr . 0x89ABCDEF)
  (resolved . 0x...)
  (plane-coordinate . 0x...)
  (origin . ...)
  (profile . ...))
```

---

6. Sparse Rank-8 Boards

The QuQuart/CONS model establishes that at rank 8:

```
CAR domain
256 byte assignments

one selected CDR
256 output bits

possible CDR population
2^256
```

The runtime stores one selected 256-bit board, not all possible boards.

A board is:

```
typedef struct {
    uint64_t lane[4];
} OmiBoard256;
```

It represents a subset:

$$B_i \subseteq P$$

Membership:

$$p \in B_i \iff \chi_i(p) = 1$$

The board is sparse conceptually even if represented as a fixed 256-bit bitset.

---

7. Disjoint Contribution Law

For naturally disjoint boards:

$$B_i \cap B_j = \varnothing \quad \text{for } i \neq j \qquad =$$

The shared visible board is:

$$B_{\text{shared}} = \bigcup_{i \in I} B_i$$

Because the boards are disjoint:

$$\left| \bigsqcup_{i \in I} B_i \right| = \sum_{i \in I} |B_i|$$

Bitwise:

```
for all i != j:

BOARD_i AND BOARD_j = 0
```

Then:

```
SHARED_BOARD
=
BOARD_0 OR BOARD_1 OR ... OR BOARD_n
```

This is the simplest blackboard composition profile.

---

8. Forced Disjointness by Tagging

Boards do not have to be spatially disjoint to participate in a coproduct.

If two sources claim the same visible position:

$$p \in B_i$$

and:

$$p \in B_j$$

the coproduct still distinguishes:

$$(i, p)$$

from:

$$(j, p)$$

This gives two levels:

```
origin-disjoint
always preserved by tagging

projection-disjoint
optional property of visible board allocation
```

This distinction is essential.

```
Coproduct disjointness
does not require different visible bytes.

Visible-plane disjointness
does require non-overlapping bitboards.
```

The runtime must record which sense is intended.

---

9. Partition of the Shared Plane

A partition of P is a family:

$$\mathcal{P} = \{P_\alpha\}_{\alpha \in A}$$

such that:

$$P_\alpha \cap P_\beta = \varnothing \quad \text{for } \alpha \neq \beta \qquad =$$

and:

$$\bigcup_{\alpha \in A} P_\alpha = P$$

The eight 32-position rectangles form one such partition of the byte plane.

Local half:

```
(0x00,0x07,0x37,0x30)
(0x08,0x0F,0x3F,0x38)
(0x40,0x47,0x77,0x70)
(0x48,0x4F,0x7F,0x78)
```

Remote/high half:

```
(0x80,0x87,0xB7,0xB0)
(0x88,0x8F,0xBF,0xB8)
(0xC0,0xC7,0xF7,0xF0)
(0xC8,0xCF,0xFF,0xF8)
```

Each rectangle contains:

$$4 \times 8 = 32$$

positions.

Together:

$$8 \times 32 = 256$$

positions.

Thus the full plane is exactly partitioned.

---

10. Omnicron Coproduct Partition

The complete construction is:

$$\mathcal{B} = \bigsqcup_{i \in I} B_i$$

followed by a projection:

$$\pi : \mathcal{B} \to P$$

The fibers of π are:

$$\pi^{-1}(p) = \{(i, p) \mid p \in B_i\}$$

A fiber may contain:

```
zero contributions

one contribution

multiple origin-distinct contributions
```

Therefore every visible blackboard coordinate has an origin fiber behind it.

This is the precise meaning of:

```
retaining the full relation behind every visible coordinate
```

The byte `0x48` is merely the displayed coordinate.

Its complete state is:

$$\pi^{-1}(0x48)$$

together with each contributor's CAR/CDR relation.

---

11. The Universal Property

The coproduct is characterized by its universal property.

For each source board B_i, there is an injection:

$$\iota_i : B_i \to \mathcal{B}$$

Suppose every source has a lawful mapping into some target blackboard X:

$$f_i : B_i \to X$$

Then there exists a unique mapping:

$$f : \mathcal{B} \to X$$

such that:

$$f \circ \iota_i = f_i$$

for every source i.

In OMI terms:

```
each .o source defines its own bounded contribution

the coproduct collects them without erasing origin

one unique resolver map combines their declared behavior
into the shared OMI-Lisp blackboard
```

This is why coproduct is a better primary abstraction than ordinary union.

---

12. CONS as the Mediating Map

Each source has a contribution function:

$$f_i : B_i \to P$$

The coproduct provides one unique mediating map:

$$CONS : \bigsqcup_i B_i \to P$$

such that:

$$CONS \circ \iota_i = f_i$$

This makes CONS the categorical blackboard compositor.

It does not erase the injections.

It does not claim that all origins are equal.

It supplies the unique shared projection consistent with all declared source maps.

Canonical statement:

```
CONS is the mediating map from the origin-tagged
coproduct of knowledge boards into the shared
OMI-Lisp coordinate plane.
```

---

13. Why CONS Is Not Plain Set Union

Ordinary union forgets source identity.

For:

$$A = \{x\}$$

and:

$$B = \{x\}$$

ordinary union gives:

$$A \cup B = \{x\}$$

The two occurrences collapse.

The coproduct gives:

$$A \sqcup B = \{(A, x), (B, x)\}$$

OMI requires the second behavior.

Therefore:

```
CONS must not be implemented as untagged OR alone
when origin identity matters.
```

Bitwise OR may produce the visible board, but the runtime must retain a side table, fiber map, tag vector, or provenance object.

---

14. Blackboard Fibers

A visible coordinate record should contain:

```c
typedef struct {
    uint8_t visible_coordinate;

    uint32_t contribution_count;
    OmiContributionRef *contributors;
} OmiBlackboardFiber;
```

Each contribution reference should bind:

```c
typedef struct {
    uint32_t source_id;

    uint32_t car;
    uint32_t cdr;

    uint16_t resolver_profile;
    uint8_t scope;
    uint8_t flags;

    uint8_t source_digest[32];
} OmiContributionRef;
```

Thus:

```
one visible byte
may expose many distinct lawful origins
```

without collapsing them.

---

15. Intersection

For two visible boards:

$$A \cap B$$

is the set of coordinates claimed by both.

Bitwise:

```
OVERLAP = A AND B
```

Intersection is not the coproduct.

It is a diagnostic of projection overlap.

Uses:

```
conflict detection

shared-interest detection

required co-presence

face or edge compatibility

scope agreement
```

The result must distinguish:

```
same visible coordinate

same semantic contribution

same origin identity
```

Those are separate predicates.

---

16. Symmetric Difference

The symmetric difference is:

$$A \triangle B = (A \setminus B) \cup (B \setminus A)$$

Bitwise:

```
A XOR B
```

It identifies positions belonging to exactly one board.

Uses:

```
contrast witness

change detection

unshared contribution surface

complementary projection

local-versus-remote difference
```

Symmetric difference is not sufficient for structural identity because it loses shared positions and is commutative.

It is a witness, not the complete CONS record.

---

17. Ordinary Union

Ordinary union is:

$$A \cup B$$

Bitwise:

```
A OR B
```

It produces the visible occupancy board.

It does not preserve:

```
which source supplied a bit

whether two sources supplied the same bit

CAR/CDR direction

resolver profile

scope

provenance
```

Therefore ordinary union is only the blackboard occupancy projection.

---

18. Tagged Union and Sum Type

A tagged union is the programming-language realization of a coproduct.

Example:

```
typedef enum {
    OMI_SOURCE_RULES,
    OMI_SOURCE_FACTS,
    OMI_SOURCE_COMBINATORS,
    OMI_SOURCE_CLOSURES,
    OMI_SOURCE_REMOTE
} OmiSourceTag;

typedef struct {
    OmiSourceTag tag;

    union {
        OmiRulesContribution rules;
        OmiFactsContribution facts;
        OmiCombinatorContribution combinator;
        OmiClosureContribution closure;
        OmiRemoteContribution remote;
    } value;
} OmiContribution;
```

This guarantees that each value belongs to one explicit case.

The categorical interpretation is:

$$Rules + Facts + Combinators + Closures + Remote$$

The plus sign here means coproduct or sum type.

The runtime must inspect the tag before interpreting the payload.

---

19. Sum Type Versus C Union

A raw C `union` stores one of several layouts but does not automatically record which member is active.

Therefore:

```
union {
    A a;
    B b;
};
```

is not by itself a safe coproduct.

A proper sum type requires:

```
tag
+
payload
```

The tag establishes the injection:

$$\iota_A : A \to A + B$$

or:

$$\iota_B : B \to A + B$$

OMI must use tagged unions, not untagged memory overlays, for authoritative contribution identity.

---

20. Disjoint Union of Graphs

Each `.o` source may contribute a graph:

$$G_i = (V_i, E_i)$$

The disjoint union of graphs is:

$$\bigsqcup_i G_i = \left(\bigsqcup_i V_i, \bigsqcup_i E_i \right)$$

Vertices and edges are origin-tagged.

Even when two graphs use the same local node ID, their coproduct vertices remain distinct:

$$(i, v) \ / \ (j, v) =$$

The OMI-Lisp blackboard may then add explicit cross-source edges after validation.

Thus there are two stages:

```
Stage 1
disjoint graph coproduct

Stage 2
validated relation edges between components
```

This is exactly how independent knowledge sources should enter a blackboard.

---

21. Topological Disjoint Union

If every source board carries its own topology:

$$(B_i, \tau_i)$$

their topological disjoint union is the coproduct topology on:

$$\bigsqcup_i B_i$$

A subset is open when its inverse image in every component is open.

For OMI, the useful analogy is:

```
each source retains its own local neighborhood law

the combined blackboard preserves those local neighborhoods
```

```
cross-source continuity is introduced only by declared mappings
```

This is valuable for:

```
local canvas layout

scope neighborhoods

Gray adjacency

quasigroup neighborhoods

projection charts

remote module boundaries
```

---

22. Direct Limits and Directed Colimits

A direct limit applies when contributions form a directed system:

$$B_0 \to B_1 \to B_2 \to \cdots$$

with compatible transition maps.

This fits incremental OMI evolution:

```
versioned source boards

successive notation expansions

Delta3 rolling windows

increasingly complete remote knowledge

progressive blackboard construction
```

The direct limit identifies compatible earlier elements with their later images.

It is not the same as the initial coproduct.

The sequence is:

```
coproduct
collect independent contributions

coequalization or validated identification
identify declared equivalents

direct limit
form the stable result of an evolving directed system
```

Thus a mature OMI blackboard may be modeled as a colimit of versioned coproduct partitions.

---

23. Colimit View of the Blackboard

Suppose knowledge sources and transition maps form a diagram:

$$D : J \to \mathbf{Set}$$

or:

$$D : J \to \mathbf{Graph}$$

The blackboard may be constructed as:

$$\operatorname{colim} D$$

Conceptually:

```
begin with all contributions disjoint

then impose only the equivalence relations
required by declared transition maps
```

This is a stronger formulation than simply merging everything.

A common construction is:

$$\operatorname{colim} D = \left(\bigsqcup_{j \in J} D(j) \right) / \sim$$

where $\sim$ is the smallest equivalence relation generated by the diagram's transition maps.

This is almost exactly the OMI blackboard process:

```
coproduct first

lawful identification second

projection third
```

---

24. Where Union-Find Belongs

Union-Find maintains a partition of elements under repeated equivalence merges.

Its operations are:

```
MakeSet(x)

Find(x)

Union(x,y)
```

This is useful after OMI has decided that two origin-tagged contributions should belong to the same equivalence class.

For example:

```
RULES.o coordinate r

FACTS.o coordinate f

validation proves r and f denote
the same resolved relation class
```

Then:

```
Union(r,f)
```

may record that equivalence.

Union-Find does not determine whether the merge is semantically valid.

It only maintains the resulting partition efficiently.

Canonical boundary:

```
Validation authorizes equivalence.

Union-Find maintains equivalence.
```

---

25. Omnicron Union-Find

An optional runtime subsystem may maintain:

```c
typedef struct {
    uint32_t parent;
    uint32_t rank;
} OmiDisjointSetNode;
```

Operations:

```c
uint32_t omi_find(
    OmiUnionFind *uf,
    uint32_t element
);

bool omi_union_validated(
    OmiUnionFind *uf,
    uint32_t left,
    uint32_t right,
    const OmiValidationWitness *witness
);
```

The API should not expose unrestricted union for authoritative state.

It should require a validation witness or validated equivalence token.

---

26. Union-Find Is Not the Blackboard Itself

Union-Find tracks equivalence classes.

It does not inherently preserve:

```
ordered CAR/CDR relations

source tags

edge direction

scope roles

board masks

projection coordinates

quasigroup profiles

temporal position

azimuth
```

Therefore the full blackboard record remains a tagged graph or coproduct object.

Union-Find is only an index over equivalence classes inside that richer structure.

---

27. Coproduct Partition Versus Union-Find

Construction	Primary question
Coproduct	How do independent sources enter one space without losing origin?
Partition	Which elements belong to distinct non-overlapping classes?
Union-Find	How do we efficiently maintain equivalence classes under validated merges?
CONS	How are ordered CAR/CDR relations resolved and projected?
Colimit	What stable object results after all declared identifications?

The architecture is:

```
Coproduct
→ Partition
→ CONS resolution
→ Validation
→ Union-Find equivalence
→ Colimit or stable blackboard
```

28. Coproduct and the Tetrahedral QuQuart

The four `.o` roots form a finite coproduct of typed contribution domains:

$$Rules \sqcup Facts \sqcup Combinators \sqcup Closures$$

The tetrahedral centroid is not another coproduct summand.

CONS is the mediating resolver across the four injections.

```
RULES.o --------
FACTS.o ---------
COMBINATORS.o -----> CONS -> shared coordinate
CLOSURES.o --------/
```

The QuQuart/CONS model identifies CONS as the tetrahedral centroid rather than a fifth state.

Categorically:

```
the vertices are input components

the centroid is the induced resolving relation
```

29. Scoped Coproduct

The four canonical scopes are:

```
FS
GS
RS
US
```

Each may contribute a scoped board:

$$B_{FS}$$

$$B_{GS}$$

$$B_{RS}$$

$$B_{US}$$

The scoped coproduct is:

$$B_{\text{scope}} = B_{FS} \sqcup B_{GS} \sqcup B_{RS} \sqcup B_{US}$$

The scoped CONS values are:

$$CONS_{FS} = (CAR_{FS}.CDR_{FS})$$

$$CONS_{GS} = (CAR_{GS}.CDR_{GS})$$

$$CONS_{RS} = (CAR_{RS}.CDR_{RS})$$

$$CONS_{US} = (CAR_{US}.CDR_{US})$$

These may form the homogeneous tetrahedral coordinate:

$$[CONS_{FS} : CONS_{GS} : CONS_{RS} : CONS_{US}]$$

The low/high `0x80` mirror remains the carrier geometry for affine and projective realization.

---

30. Affine and Projective Blackboard Realization

The projective blackboard retains all four scoped components:

$$P = [CONS_{FS} : CONS_{GS} : CONS_{RS} : CONS_{US}]$$

A selected affine realization is:

```
(omi---imo . ΔUS/?O_oΔ)
```

The Tangential Gauge makes readable notation reachable by crossing from the hidden control hinge through `SP` into the printable branch. The low Omicron doctrine treats this traversal as reachability rather than a mere parser permission.

The coproduct interpretation is:

```
projective blackboard
retains all origin-tagged scoped contributions

affine chart
selects one readable realization

centroid
```

```
preserves the shared relation

projection
does not erase the coproduct fibers
```

---

31. Character Encoding Must Be Earned

OMI-Lisp does not begin with arbitrary printable symbols.

The low gauge first establishes reachability:

```
0x00..0x1F
hidden control

0x1C..0x1F
FS/GS/RS/US control hinge

0x1F
US tangent

0x20
SP readable boundary

0x2F..0x6F
printable F-column branch
```

The character ring doctrine states that dot notation becomes reachable only after the gauge traverses into readable space.

Thus the encoding pipeline is:

```
origin-tagged source contribution

→ scoped control coordinate

→ Tangential Gauge traversal

→ readable character coordinate

→ dotted OMI-Lisp declaration
```

```
→ CONS blackboard projection
```

The character is earned because it is the result of a lawful route through the gauge, not merely because a parser accepts its byte.

---

32. The OMI-Lisp Character as a Coproduct Image

Let:

$$\chi_i : B_i \to P$$

be the character projection for source i.

The coproduct induces:

$$\chi : \bigsqcup_i B_i \to P$$

A visible character coordinate:

$$p \in P$$

may therefore be the image of many origin-tagged elements:

$$(i_1, b_1)$$

$$(i_2, b_2)$$

$$\dots$$

This explains how independent remote knowledge sources may share one visible OMI-Lisp character plane without collapsing.

The character is common.

The fibers retain difference.

---

33. Character Ring and Place-Value Cascade

The low Omicron character ring has:

```
128 positions

four 32-position sectors
```

The outer place-value cascade has:

```
16 hexadecimal lanes
```

A position is:

$$position = (lane \ll 12) \mid character$$

The sign and its place remain distinct:

```
character
selects the sign role

lane
selects addressed interpretation
```

The same visible character may appear in multiple place-value lanes without becoming the same full coordinate.

This mirrors the coproduct principle:

```
same visible sign

different origin or place

distinct full element
```

---

34. Graph Blackboard Structure

A complete OMI blackboard should be modeled as a graph:

$$G = (V, E)$$

Vertices include:

```
source contributions

CAR coordinates

CDR continuations

CONS results

visible plane coordinates

scope nodes

module nodes

validation nodes
```

Edges include:

```
contributed-by

carried-by

continued-by

resolved-to

projected-to

equivalent-to

validated-by

mirrored-by
```

The initial graph is a disjoint union of source graphs.

Validated cross-source edges are added afterward.

---

35. OMI-Lisp Representation

```
(blackboard
  (plane . omi-lisp-256)
```

```
  (coproduct .
    ((inject
       (origin . RULES.o)
       (car . 0x12345678)
       (cdr . 0x89ABCDEF)
       (board . #x...))

     (inject
       (origin . FACTS.o)
       (car . 0x11223344)
       (cdr . 0x55667788)
       (board . #x...))))

  (cons .
    (profile . ordered-quasigroup)
    (projection . omi-plane-256))

  (partition .
    (local-control
     local-readable
     projective-control
     projective-readable))

  (equivalence .
    (strategy . validated-union-find)))
```

---

36. C Runtime Model

```
typedef struct {
    uint32_t source_id;

    uint32_t car;
    uint32_t cdr;

    OmiBoard256 board;

    uint16_t resolver_profile;
    uint8_t scope;
    uint8_t contribution_type;

    uint8_t digest[32];
} OmiBoardContribution;
```

```
typedef struct {
    OmiBoard256 occupancy;
    OmiBoard256 conflict;

    OmiBlackboardFiber fibers[256];

    OmiUnionFind equivalence;

    uint32_t contribution_count;
} OmiCoproductBlackboard;
```

---

37. Injection

```
omi_status omi_blackboard_inject(
    OmiCoproductBlackboard *blackboard,
    const OmiBoardContribution *contribution
);
```

Injection must:

```
preserve source identity

preserve CAR/CDR order

copy or retain the board safely

add each active position to its fiber

update visible occupancy

record overlaps without collapsing them
```

---

38. CONS Composition

```
omi_status omi_blackboard_cons(
    OmiCoproductBlackboard *blackboard,
    uint32_t left_contribution,
    uint32_t right_contribution,
```

```
    uint16_t resolver_profile,
    OmiConsResult *out
);
```

The result should include:

```
ordered inputs

resolved full word

visible board

visible coordinates

origin fibers

overlap board

conflict status

recovery metadata
```

---

39. Validation and Equivalence

```
omi_status omi_blackboard_validate_equivalence(
    OmiCoproductBlackboard *blackboard,
    OmiContributionRef left,
    OmiContributionRef right,
    OmiValidationResult *out
);
```

Only after successful validation:

```
omi_status omi_blackboard_union(
    OmiCoproductBlackboard *blackboard,
    OmiContributionRef left,
    OmiContributionRef right,
    const OmiValidationResult *validation
);
```

This enforces:

```
no merge without law

no equivalence without witness

no loss of origin history
```

---

40. Direct-Limit Evolution

Versioned blackboards may form:

$$B_0 \to B_1 \to B_2 \to \cdots$$

Each transition must preserve previously accepted coordinates or explicitly supersede them under a version law.

The stable evolving object is the direct limit:

$$B_\infty = \varinjlim B_n$$

Operationally:

```
append contribution

preserve injection

add validated equivalence

retain ancestry

compute current projection
```

The direct limit is the stable accumulated blackboard, not a destructive overwrite.

---

41. Relationship to Listed Constructions

Coproduct

Primary origin-preserving composition.

```
many independent types or sets

one sum object

explicit injections

no forced identification
```

Direct limit

Stable result of a directed sequence of compatible blackboards.

```
versions

temporal accumulation

progressive resolution
```

Topological disjoint union

Preserves each source's local neighborhood and projection topology.

```
local layouts

chart neighborhoods

remote module boundaries
```

Disjoint union of graphs

Combines independent source graphs before cross-source relations are added.

```
source graph coproduct

then validated linking
```

Intersection

Detects jointly occupied visible positions.

```
overlap

agreement candidate

conflict candidate
```

Set identities

Supply algebraic simplification and conformance laws.

Examples:

$$A \cup \varnothing = A$$

$$A \cap \varnothing = \varnothing$$

$$A \triangle A = \varnothing$$

$$A \triangle \varnothing = A$$

Partition

Defines mutually exclusive visible regions or equivalence classes.

```
plane sectors

scope classes

validated identity classes
```

Sum type

Programming-language realization of a coproduct.

```
one tagged case among many
```

Symmetric difference

Shows positions belonging to exactly one contributor.

```
contrast surface

change surface
```

Tagged union

Concrete storage form preserving which contribution variant is active.

```
tag + payload
```

Raw union type

Memory-sharing implementation device.

It becomes authoritative only when paired with a valid active-member tag.

---

42. Set Identities for OMI Boards

For boards A, B, C:

Commutativity of visible union

$$A \cup B = B \cup A$$

This applies to occupancy projection.

It does not imply CAR/CDR commutativity.

Associativity of visible union

$$(A \cup B) \cup C = A \cup (B \cup C)$$

Again, this applies to occupancy only.

Distributivity

$$A \cap (B \cup C) = (A \cap B) \cup (A \cap C)$$

Useful for scoped overlap tests.

Symmetric difference

$$A \triangle B = B \triangle A$$

Useful as contrast, not ordered identity.

Disjoint union cardinality

If:

$$A \cap B = \varnothing$$

then:

$$|A \sqcup B| = |A| + |B|$$

---

43. Ordered Relation Versus Commutative Board Algebra

The bitboard operations:

```
OR

AND

XOR
```

are commutative.

The OMI relation:

```
(CAR . CDR)
```

is ordered.

Therefore the runtime must maintain two layers:

```
board occupancy algebra
may be commutative
```

```
relation resolution algebra
must preserve CAR/CDR order
```

The ordered quasigroup construction described in the QuQuart/CONS model provides reversible non-collapsing CAR/CDR resolution.

---

44. Coproduct Versus Quasigroup

These solve different problems.

```
Coproduct
preserves independent origins

Quasigroup
provides reversible ordered resolution
```

The combined structure is:

$$\coprod_i B_i \xrightarrow{CONS} P$$

where CONS uses a selected quasigroup law to resolve relations.

A coproduct alone does not compute a relation.

A quasigroup alone does not preserve distributed origin.

Together they provide the blackboard architecture.

---

45. Coproduct Versus Centroid

The tetrahedral vertices are independent typed contributions.

Their coproduct-like injection preserves each role:

```
RULES

FACTS

COMBINATORS
```

```
CLOSURES
```

The centroid CONS is not another injected component.

It is the induced relation that coordinates them.

Thus:

```
coproduct
collects the vertices

centroid
resolves their common relation
```

---

46. The Four Scope Boards

Each scope may maintain one board:

```
FS board
frame/file contribution positions

GS board
group/graph contribution positions

RS board
record/relation contribution positions

US board
unit/symbol contribution positions
```

The scope coproduct is:

$$B_{FS} \sqcup B_{GS} \sqcup B_{RS} \sqcup B_{US}$$

CONS may produce:

```
one shared occupancy board

four scoped fibers
```

```
one homogeneous tetrahedral coordinate

one affine readable realization
```

---

47. Low and High Rails

The locked 256-bit character frame is:

```
LOW 128
2 4 6 Δ

HIGH 128
1 3 5 Δ
```

Logical presentation:

```
Δ 1 2 3 4 5 6 Δ
```

The low/high rails are physical storage partitions.

The logical ring is a presentation order.

The coproduct model allows contributors on both rails to remain origin-distinct while sharing the logical ring.

---

48. Gnomonic Projection and Azimuth

After a relation is resolved and validated, the Gnomonic Projection Azimuth orients it over the byte circle:

$$0x00..0xFF$$

The azimuth is observer-relative projection metadata, not validation or identity authority.

The coproduct state remains intact beneath the projection.

```
coproduct preserves source

CONS resolves relation
```

```
validation classifies

azimuth orients display
```

---

49. Delta3 and Integrity

Scope, Hamming integrity, temporal position, and projection orientation remain separate dimensions.

The Delta3/Hamming doctrine defines:

```
FS/GS/RS/US
scope

LOGOS/NOMOS/PATHOS
integrity

LL/MM/NN
temporal position

azimuth
observer-relative orientation
```

These axes may annotate a contribution without changing its coproduct injection.

A complete contribution may therefore carry:

```
origin tag

CAR/CDR

scope

Hamming status

Delta position

azimuth

board

resolver profile
```

50. Earned Character Encoding

A source does not gain a printable OMI-Lisp coordinate merely by transmitting bytes.

It must pass through:

```
1. source injection

2. scope assignment

3. control-plane placement

4. Tangential Gauge reachability

5. SP crossing

6. printable branch selection

7. character/place resolution

8. CONS composition

9. structural validation
```

Only then does the source obtain a readable OMI-Lisp character coordinate.

Canonical statement:

```
The character is the projected image
of a lawfully injected and resolved contribution.
```

51. Independent and Remote Sources

A remote source may submit:

```
source ID

module digest
```

```
CAR

CDR

scope

board

resolver profile

signature or witness

version ancestry
```

The blackboard first injects the contribution disjointly.

It does not immediately merge it with local state.

The stages are:

```
receive

tag

inject

inspect

resolve

validate

identify if lawful

project
```

This is safer and more mathematically faithful than direct mutation of shared state.

---

52. Conflict Handling

When two origin-distinct contributions project to one coordinate:

```
do not collapse

do not overwrite

do not union automatically
```

Instead record:

```
fiber cardinality

source identities

intersection class

resolver compatibility

validation status
```

Possible states:

```
parallel

compatible

complementary

conflicting

equivalent

unresolved
```

Union-Find may merge only the `equivalent` case.

---

53. Blackboard Acceptance Law

A visible coordinate p is eligible for accepted projection only when:

```
its contribution exists in the coproduct

its source tag is valid
```

```
its CAR/CDR relation is well-formed

its resolver profile is available

its scope is valid

its character route is reachable

its relevant edge/face predicates pass

its projection does not erase required provenance
```

Acceptance does not delete rejected or unresolved contributions from the coproduct.

They remain origin-tagged members with a different status.

---

54. Canonical Runtime Sequence

```
1. Create one origin tag for each .o source.

2. Parse its declared CAR/CDR relation.

3. Load its selected 256-bit board.

4. Inject the contribution into the coproduct.

5. Record all active byte coordinates in origin fibers.

6. Preserve overlaps without collapsing them.

7. Resolve selected ordered pairs through CONS.

8. Generate the visible occupancy board.

9. Apply scope and tetrahedral consistency checks.

10. Traverse the Tangential Gauge when readable notation is required.

11. Produce the affine OMI-Lisp character projection.

12. Retain the projective scoped tuple.

13. Validate candidate equivalences.
```

```
14. Apply Union-Find only to validated equivalent elements.

15. Preserve graph ancestry and source tags.

16. Advance versioned systems through directed transition maps.

17. Form the current colimit view without destroying the original injections.
```

---

55. Canonical Terminology

```
Omnicron Coproduct
the origin-tagged sum of all knowledge contributions
```

```
Omnicron Partition
the mutually exclusive region or equivalence-class structure
```

```
Omnicron Coproduct Partition
the complete origin-preserving blackboard structure
```

```
OMI-Lisp Plane
the shared 256-position visible projection surface
```

```
Contribution Board
one selected 256-bit source predicate
```

```
Blackboard Fiber
all origin-tagged contributions projecting to one visible coordinate
```

```
CONS
the ordered resolver and categorical mediating map
```

```
Union-Find
the optional validated equivalence-class maintenance structure
```

```
Direct Limit
the stable result of a directed sequence of compatible blackboard versions
```

---

56. One-Line Canon

The Omnicron Coproduct Partition is the origin-preserving sum of independent local or remote `.o` knowledge boards: each source injects a tagged sparse 256-bit predicate together with its ordered 32-bit CAR/CDR relation; CONS acts as the unique mediating resolver into the shared 256-position OMI-Lisp plane, while every visible coordinate retains a fiber of its full source, scope, resolver, and relation provenance; validation alone may identify contributions as equivalent, Union-Find maintains those validated equivalence classes, and directed colimits preserve the evolving blackboard without erasing the original coproduct injections.

---

57. Final Canon Lock

```
Independent knowledge sources enter by coproduct.

They do not enter by destructive merge.
```

```
Every contribution is tagged by origin.
```

```
One selected rank-8 contribution board is 256 bits.
```

```
The shared OMI-Lisp plane has 256 visible coordinates.
```

```
A visible coordinate may have multiple
origin-distinct contributions behind it.
```

```
The full relation retains:

ordered CAR
ordered CDR
source
scope
```

```
resolver
board
and carrier profile.
```

```
CONS resolves and projects.

CONS does not erase origin.
```

```
Bitwise OR produces occupancy.

It does not produce full identity.
```

```
Intersection detects overlap.

Symmetric difference detects exclusive contribution.

Neither replaces the coproduct record.
```

```
Tagged unions implement coproducts.

Raw untagged unions do not.
```

```
A graph blackboard begins as
a disjoint union of source graphs.
```

```
Cross-source edges are added only
through declared and validated relations.
```

```
Union-Find does not decide equivalence.

Validation decides equivalence.

Union-Find maintains it.
```

```
A direct limit describes the stable result
of a directed sequence of compatible blackboards.
```

```
The low/high byte carrier remains
the exact XOR-0x80 mirror.
```

```
The character ring makes notation reachable.

The coproduct preserves source difference.

CONS composes the shared board.

The OMI-Lisp character is earned
as the readable projection of a lawfully
injected, resolved, and retained relation.
```